Spillekvelden med ProtectedLocalStorage

Ombygging av `GameNightState` slik at spillekvelden overlever refresh og lukket fane, pluss en knapp som nullstiller spillekvelden.

> **Grønn markering** = kode som er lagt til eller endret på grunn av ProtectedLocalStorage. Umarkert kode er som før, eller hører til nullstill-funksjonen.

Hva endres

Fil	Endring
`GameNightState.cs`	All lagringskode samlet i én region, «Browser storage». De offentlige metodene kaller bare `SaveAsync()`
`MainLayout.razor`	Laster spillekvelden én gang når appen er koblet til
`Games.razor`	Bruker de asynkrone metodene og abonnerer på `OnChange`
`GamesDetails.razor`	Bruker de asynkrone metodene og abonnerer på `OnChange`
`GameNight.razor`	Abonnerer på `OnChange` og får knapp for å nullstille
`Program.cs`	Ingen endring, fortsatt `AddScoped<GameNightState>()`

Hvorfor

En scoped tjeneste lever bare så lenge circuiten lever. Ved refresh lages en ny circuit og en ny, tom `GameNightState`. For at spillekvelden skal overleve, må den lagres utenfor serverens minne. `ProtectedLocalStorage` lagrer krypterte data i nettleserens localStorage, så de overlever både refresh og lukket fane.

1. GameNightState.cs

All kode som har med nettleserlagringen å gjøre, ligger samlet nederst i regionen **Browser storage**. Resten av klassen vet bare at den skal kalle `SaveAsync()` etter en endring. Vil du senere bytte til `ProtectedSessionStorage` eller et API, er det bare den regionen som må endres.

GameNightState.cs

```
using System.Security.Cryptography;
using AK2.Models;
using Microsoft.AspNetCore.Components.Server.ProtectedBrowserStorage;

namespace AK2.Components.Shared
{
    public class GameNightState(ProtectedLocalStorage storage)
    {
        private readonly List<BoardGame> games = new();

        public IReadOnlyList<BoardGame> Games => games;
        public int Count => games.Count;
        public int TotalPlayTime => games.Sum(g => g.PlayTimeMinutes);
```

```
        // The bell
        public event Action? OnChange;

        public bool Contains(int gameId) => games.Any(g => g.Id == gameId);

        public async Task<bool> AddAsync(BoardGame game)
        {
            // Early return if BoardGame already exists
            if (Contains(game.Id)) return false;

            games.Add(game);
            await SaveAsync();

            // Ring the bell
            OnChange?.Invoke();
            return true;
        }

        public async Task RemoveAsync(int gameId)
        {
            games.RemoveAll(g => g.Id == gameId);
            await SaveAsync();
            OnChange?.Invoke();
        }

        // Resets the game night
        public async Task ClearAsync()
        {
            games.Clear();
            await SaveAsync();
            OnChange?.Invoke();
        }

        #region Browser storage (ProtectedLocalStorage)

        private const string StorageKey = "gameNight";
        private bool loaded;

        // Loads the game night from the browser once per circuit.
        // Called from MainLayout after the first render.
        public async Task LoadAsync()
        {
            if (loaded) return;
            loaded = true;

            try
            {
                var result = await storage.GetAsync<List<BoardGame>>(StorageKey);

                if (result.Success && result.Value is not null)
                {
                    games.Clear();
                    games.AddRange(result.Value);
                    OnChange?.Invoke();
                }
            }
            catch (CryptographicException)
            {
                // Stored data can no longer be decrypted, start fresh
                await storage.DeleteAsync(StorageKey);
```

```
            }
        }

        // Saves the current list, or removes the key when the list is empty
        private async Task SaveAsync()
        {
            if (games.Count == 0)
                await storage.DeleteAsync(StorageKey);
            else
                await storage.SetAsync(StorageKey, games);
        }

        #endregion
    }
}
```

Om lagringskoden

- `ProtectedLocalStorage` injiseres i konstruktøren. Den er innebygd i Blazor Server og trenger ingen egen registrering.
- Lagring i nettleseren går via JavaScript og er asynkron. Derfor er `Add` og `Remove` blitt `AddAsync` og `RemoveAsync`.
- `loaded` hindrer at spillene lastes flere ganger i samme circuit.
- `CryptographicException` kan oppstå hvis krypteringsnøklene på serveren er endret siden dataene ble lagret. Da slettes de gamle dataene.
- `SaveAsync` sletter nøkkelen når listen er tom. Derfor tømmer `ClearAsync` også lagringen, så spillene ikke kommer tilbake ved neste refresh.
- `#region` er bare for oversikt. Visual Studio lar deg folde regionen sammen.

2. MainLayout.razor

Nettleserlagring kan ikke leses under prerendering, bare etter at siden er koblet til. Derfor lastes spillekvelden i `OnAfterRenderAsync`. `MainLayout` vises alltid, så lastingen skjer uansett hvilken side brukeren åpner.

`MainLayout.razor (legg til øverst og i @code)`

```
@inherits LayoutComponentBase
@inject GameNightState GameNight

@* ...eksisterende markup uendret... *@

@code {
    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        if (firstRender)
            await GameNight.LoadAsync();
    }
}
```

3. Games.razor

To endringer: `HandleAdd` og `HandleRemove` bruker de asynkrone metodene, og siden abonnerer på `OnChange`. Abonnementet er nødvendig fordi spillene lastes inn *etter* at siden er tegnet første gang. Uten det ville kortene ikke vist «Valgt» før brukeren klikket på noe.

Games.razor (endrede deler)

```
@page "/games"
@inject Data.GamesStorageAndQuery gamesQuery
@inject GameNightState GameNight
@implements IDisposable

@* ...markup uendret... *@

@code {
    // ...eksisterende felt og metoder...

    protected override void OnInitialized()
    {
        ApplyFilter();
        GameNight.OnChange += OnGameNightChanged;
    }

    private void OnGameNightChanged() => InvokeAsync(StateHasChanged);

    private async Task HandleAdd(BoardGame game)
    {
        messages[game.Id] = await GameNight.AddAsync(game)
            ? "Spillet ble lagt til"
            : "Spillet er allerede lagt til";

        await Task.Delay(3000);
        messages.Remove(game.Id);
    }

    private async Task HandleRemove(BoardGame game)
        => await GameNight.RemoveAsync(game.Id);

    public void Dispose() => GameNight.OnChange -= OnGameNightChanged;
}
```

4. GamesDetails.razor

Samme mønster som Games-siden.

GamesDetails.razor (endrede deler)

```
@inject GameNightState GameNight
@implements IDisposable

@code {
    // ...eksisterende felt og metoder...

    protected override void OnInitialized()
        => GameNight.OnChange += OnGameNightChanged;

    private void OnGameNightChanged() => InvokeAsync(StateHasChanged);

    private async Task HandleAdd(BoardGame g)
    {
        message = await GameNight.AddAsync(g)
            ? "Spillet ble lagt til"
            : "Spillet er allerede lagt til";

        await Task.Delay(3000);
        message = null;
    }

    private async Task HandleRemove(BoardGame g)
        => await GameNight.RemoveAsync(g.Id);

    public void Dispose() => GameNight.OnChange -= OnGameNightChanged;
}
```

Har du allerede en `OnInitialized` eller `OnParametersSetAsync` som henter spillet, legger du bare abonnementet til i den.

5. GameNight.razor med nullstill-knapp

Knappen spør om bekreftelse før spillekvelden tømmes, siden handlingen ikke kan angres. Den er deaktivert når ingen spill er valgt. Selve knappen er ny funksjonalitet og derfor ikke grønn. Abonnementet er grønt fordi det trengs på grunn av lasting etter første tegning.

GameNight.razor

```
@page "/game-night"
@inject GameNightState GameNight
@implements IDisposable

<h1>Spillkveld</h1>
<h3>Valgte spill:</h3>

@if (GameNight.Count == 0)
{
    <p>Ingen spill valgt ennå.</p>
}
else
{
    @foreach (var game in GameNight.Games)
    {
        <p>@game.Name</p>
    }
}
```

```
<p>Total spilletid: @GameNight.TotalPlayTime minutter</p>
<p>Antall spill: @GameNight.Count</p>

@if (confirmReset)
{
    <p>Er du sikker på at du vil nullstille spillekvelden?</p>
    <button class="btn btn-danger" @onclick="ResetGameNight">Ja, nullstill</button>
    <button class="btn btn-secondary" @onclick="() => confirmReset = false">Avbryt</button>
}
else
{
    <button class="btn btn-outline-danger"
            disabled="@(GameNight.Count == 0)"
            @onclick="() => confirmReset = true">
        Nullstill spillekvelden
    </button>
}

<p><a href="/">Til forside</a></p>

@code {
    private bool confirmReset;

    protected override void OnInitialized()
        => GameNight.OnChange += OnGameNightChanged;

    private void OnGameNightChanged() => InvokeAsync(StateHasChanged);

    private async Task ResetGameNight()
    {
        await GameNight.ClearAsync();
        confirmReset = false;
    }

    public void Dispose() => GameNight.OnChange -= OnGameNightChanged;
}
```

- `confirmReset` er lokal tilstand. Den gjelder bare denne siden.
- `ClearAsync` kaller `OnChange`, så badgen i nav-menyen går til 0 med en gang.

6. Program.cs

Ingen endring. `GameNightState` skal fortsatt være scoped. Det er fortsatt én instans per bruker, nå med nettleseren som varig lager.

Program.cs

```
builder.Services.AddScoped<GameNightState>();
```

Slik henger det sammen

Flyt

```
Oppstart / refresh
  MainLayout.OnAfterRenderAsync(firstRender)
    → GameNight.LoadAsync()
      → ProtectedLocalStorage.GetAsync("gameNight")
      → OnChange  → badge, Games, GamesDetails og GameNight tegnes på nytt

Legg til / fjern
  GameNightButton → OnAdd / OnRemove (EventCallback)
    → forelderens HandleAdd / HandleRemove
      → GameNight.AddAsync / RemoveAsync
        → liste i minnet
        → SaveAsync → ProtectedLocalStorage.SetAsync
        → OnChange

Nullstill
  GameNight.razor → ResetGameNight()
    → GameNight.ClearAsync()
      → tom liste
      → SaveAsync → ProtectedLocalStorage.DeleteAsync
      → OnChange
```

Godt å vite

Om ProtectedLocalStorage

- Badgen kan vise 0 et øyeblikk etter refresh før spillene er lastet. Det er normalt.
- Bytt `ProtectedLocalStorage` med `ProtectedSessionStorage` hvis spillekvelden skal glemmes når fanen lukkes. Det er bare typen i konstruktøren og using-linjen som endres.
- Lagringen er per nettleser. Bruker samme person en annen PC eller nettleser, er spillekvelden tom der.
- Det lagres hele spillobjekter. Når spillene senere hentes fra API-et, kan det være ryddigere å lagre bare ID-ene og slå opp spillene ved lasting. Da er det bare regionen «Browser storage» som må endres.

- Søk etter andre steder som kaller `GameNight.Add(` eller `GameNight.Remove(`. De må også byttes til de asynkrone metodene, ellers får du kompileringsfeil.

Forslag til README

«`GameNightState` er registrert som Scoped, så hver bruker har sin egen spillekveld. For at den skal overleve refresh og lukket fane, lagres den med `ProtectedLocalStorage` i nettleseren. All lagringskode er samlet i én region i `GameNightState`, slik at resten av appen ikke trenger å vite hvordan spillekvelden lagres. Den lastes én gang i `MainLayout` etter første tegning, fordi nettleserlagring ikke er tilgjengelig under prerendering. Komponenter som viser spillekvelden, abonnerer på `OnChange` og melder seg av i `Dispose`.»